Spring Cloud Config Server exposes properties through a series of endpoints using different combinations of the `{application}`, `{profile}`, and `{label}` parameters:

```
/{application}/{profile}[/{label}]
/{application}-{profile}.yml
/{label}/{application}-{profile}.yml
/{application}-{profile}.properties
/{label}/{application}-{profile}.properties
```

You won't need to call these endpoints from your application when using Spring Cloud Config Client (it does that for you), but it's useful to know how the server exposes configuration data. A configuration server built with Spring Cloud Config Server exposes a standard REST API that any application can access over a network. You can use the same server for applications built with other languages and frameworks and use the REST API directly.

In chapter 14, I'll address more aspects of how to handle configuration. For example, Spring Cloud Config has a few features for encrypting properties containing secrets before storing them in a Git repository. Also, multiple backend solutions can be used as configuration data repositories, meaning that you could save all the non-sensitive properties in Git and use HashiCorp Vault to store secrets. Furthermore, the REST API itself should be protected, and I'll talk about that as well. I will address all those critical aspects from a security point of view—considering these issues is necessary before deploying to production.

For now, let's complete our solution and update Catalog Service to integrate with the Config Service application.